

CTG GO — Repositorio y Guía para Agentes IA

MVP v0.1 — Estructura propuesta y reglas de implementación

1. Propósito y lectura obligatoria

Este documento define dónde debe vivir cada componente y cómo trabajar con el contexto del proyecto. Es una especificación del futuro repositorio; no se creó código ni infraestructura en esta entrega.

Orden de lectura: Requerimientos Funcionales y Casos de Uso — MVP v0.1; Arquitectura Interna del Backend — MVP v0.1; esta guía; Arquitectura y Stack Tecnológico — MVP v0.1 como propuesta tecnológica. Consultar el Entendimiento Funcional para antecedentes. La propuesta entre fundadores no añade funcionalidades ni permisos al sistema.

Los acuerdos explícitos del usuario gobiernan el alcance. Los mecanismos de arquitectura son propuestas hasta su revisión; no confundir un borrador documental con una decisión aprobada. Si un agente encuentra contradicciones debe indicar las secciones afectadas y resolverlas con el responsable antes de implementar la parte ambigua.

2. Monorepo propuesto

README.md: entrada al proyecto, objetivo, enlaces/documentos vigentes y comandos reales de desarrollo cuando existan.

AGENTS.md: reglas breves para agentes, restricciones de negocio, lectura previa y validaciones exigidas. Debe reflejar la sección 6 de esta guía.

backend/pyproject.toml y backend/uv.lock: paquete Python y dependencias fijadas.

backend/src/ctg_go/: código importable. backend/tests/: pruebas Python.

backend/migrations/: revisiones Alembic.

frontend/package.json y frontend/package-lock.json: dependencias frontend; propuesta de usar npm y un solo lockfile. frontend/src/: aplicación React/TypeScript. frontend/tests/: pruebas propias del frontend.

deploy/: Dockerfiles, compose.yaml, configuración específica de staging/producción y proxy. scripts/: automatizaciones pequeñas, con propósito y manejo de errores.

docs/: copias versionadas de las decisiones, contratos y guías operativas. tests/e2e/: recorridos completos con Playwright. .env.example: nombres y valores ficticios; nunca secretos reales.

No crear una carpeta por cada hipótesis futura. No agregar WhatsApp, OCR, facturación ni arquitectura multiempresa completa en el esqueleto inicial.

3. Organización del backend

backend/src/ctg_go/main.py: creación de la aplicación. bootstrap.py: composición de dependencias y adaptadores. settings.py: configuración validada. api/: router raíz y traducción uniforme de errores; sin reglas de negocio.

backend/src/ctg_go/modules/: identity, clients, masters, gestions, cpe, assistant, arca, sisa, alerts, documents, audit y dashboard. Cada módulo se crea al iniciar su funcionalidad, no como colección de carpetas vacías.

Dentro de un módulo: router.py para HTTP; schemas.py para contratos Pydantic; use_cases.py para coordinación; domain.py para reglas/estados; ports.py para contratos externos necesarios; persistence.py para consultas y modelos del módulo; tasks.py solo si recibe trabajos. Separar archivos grandes por caso de uso cuando el tamaño lo justifique. Ejemplo de ubicación: modules/cpe/use_cases.py contiene ReassignCPE y RequestOfficialOperation; modules/cpe/domain.py contiene reglas de responsable y transiciones; modules/arca/adapters.py implementa llamadas oficiales; modules/cpe/tasks.py invoca el caso de uso de procesamiento. Nunca llamar al adaptador ARCA desde un endpoint o prompt saltándose la autorización.

backend/src/ctg_go/infrastructure/: conexión/transacciones, configuración de Celery, outbox, almacenamiento y logging. shared/: únicamente tipos transversales pequeños, contexto de actor y errores comunes. Prohibido convertir shared/ o utils/ en depósito de lógica de negocio.

El dominio no depende de transportes. Los adaptadores dependen de puertos definidos por sus consumidores. Los módulos exponen casos de uso/DTOs, no acceso de escritura a sus tablas internas. No usar herencia genérica de CRUD que permita modificar campos sensibles.

4. Organización del frontend

frontend/src/app/: rutas, proveedores y composición. features/: auth, clients, gestions, cpe y dashboard según se implementen. components/ui/: componentes reutilizables. lib/api/: cliente y tipos generados desde OpenAPI. lib/: utilidades técnicas acotadas.

El frontend presenta permisos y estados recibidos del backend. Ocultar un botón mejora la experiencia, pero no es un control de seguridad. No copia reglas oficiales ni decide que una CPE quedó emitida porque el servidor devolvió 202.

No editar a mano los archivos generados. Registrar el comando de generación y verificar que contrato backend y cliente coincidan. Una sola biblioteca de componentes y convenciones compartidas para formularios, errores y estados de carga.

5. Documentación dentro del futuro repositorio

docs/README.md: índice con título, versión y estado de cada documento.

docs/requirements/mvp.md: requisitos. docs/architecture/backend.md: arquitectura interna.

docs/architecture/stack.md: selección tecnológica. docs/architecture/repository.md: esta

guía. docs/adr/: decisiones específicas con contexto, alternativas, decisión, consecuencias y estado.

docs/integrations/arca.md y sisa.md: fuentes oficiales verificadas, capacidades confirmadas y pruebas de homologación. docs/runbooks/: despliegue, restauración y resolución de operaciones inciertas. docs/contracts/: contrato OpenAPI generado cuando exista. Al crear el repositorio, incorporar una copia Markdown revisada de los documentos y registrar identificador/versión de origen. Drive permite revisión de negocio; la copia versionada permite reproducir qué especificación utilizó un agente para cada commit. No asumir sincronización automática: los cambios se concilian explícitamente. Un agente sin acceso a Drive debe recibir las copias pertinentes antes de implementar. No debe reconstruir reglas de memoria ni afirmar que leyó documentación inaccesible.

6. Contenido normativo para AGENTS.md

Antes de editar: leer docs/README.md, requisitos, arquitectura y decisiones aplicables; identificar el objetivo y los criterios de aceptación. Consultar el código existente antes de agregar dependencias o abstraer.

Preservar: una organización; consulta compartida de CPE; mutación solo por responsable; administrador reasigna antes de intervenir; creador inmutable; auditoría; confirmación humana; independencia de Gestión/CPE; contexto aislado; SISA informativo salvo regla oficial verificada.

Toda escritura verifica ámbito de organización, actor habilitado, permisos, estado y versión. No confiar en organization_id/owner_id enviados libremente por el cliente. Procesar tareas bajo el contexto validado, conservando al actor original y verificando autorización vigente.

La IA de producto solo interpreta y prepara borradores. No dispone de SQL arbitrario, credenciales oficiales ni herramientas para operaciones ARCA. Un agente de desarrollo no debe desactivar estas barreras para simplificar pruebas.

No inventar métodos ARCA, estados oficiales, validaciones fiscales o reglas de negocio. Distinguir simulador de implementación real. No reintentar a ciegas una operación con resultado incierto.

No introducir nuevas tecnologías, módulos fuera de alcance o cambios de permisos como efecto secundario de una tarea. Exponer decisiones pendientes y bloquear solo la parte afectada cuando sea necesario.

No incluir secretos ni datos reales sensibles en repositorio, fixtures, prompts, logs o capturas. Usar datos sintéticos en pruebas. Separar el conocimiento profesional general de activos propietarios de empleadores o terceros.

Al cerrar una tarea: informar qué cambió, qué reglas se preservaron, pruebas ejecutadas y limitaciones. Actualizar documentación afectada y no declarar completado un flujo que depende de mocks sin indicarlo. No afirmar pruebas no ejecutadas.

7. Pruebas y validación continua

backend/tests/unit/: permisos, transiciones y reglas puras. integration/: transacciones, constraints e idempotencia con PostgreSQL real de prueba. contracts/: normalización de proveedores con fixtures saneadas. tests/e2e/: recorridos de operador y administrador. Casos esenciales: lectura compartida y edición denegada; reasignación auditada; bloqueo por operación incierta; duplicación de entrega sin doble emisión; conflicto de versión; lote

con resultado parcial; respuesta IA tardía; cierre de Gestión independiente; ingesta SISA incompleta.

CI propuesta: formato/lint y tipos Python; pruebas unitarias e integración relevantes; tipos/lint/build frontend; verificación del cliente OpenAPI; E2E de recorridos críticos antes de liberar; construcción de imágenes versionadas. Los comandos exactos se documentan cuando el repositorio exista; no se presentan aquí como ejecutables ya disponibles.

No exigir porcentaje arbitrario de cobertura ni pruebas que solo repitan la implementación. Priorizar invariantes, permisos y recuperación ante fallos. Las pruebas normales no deben llamar a producción ARCA.

8. Despliegue y configuración

Una imagen backend para API y workers con comandos distintos. Dependencias fijadas; imágenes sin latest como única referencia. Procesos no root, healthchecks, redes privadas para base/cola y secretos inyectados fuera del código.

Migraciones como paso único controlado. No ejecutar cambios de esquema al iniciar cada worker. Definir compatibilidad y estrategia de recuperación de datos; revertir imagen no revierte automáticamente una migración.

Logs con correlation_id, operation_id y contexto mínimo necesario. Auditar cambios de responsabilidad y confirmaciones. Backups fuera del host con restauración probada. La configuración de desarrollo no habilita certificados ni endpoints de producción por defecto.

9. Secuencia propuesta de implementación

Incremento 1: repositorio, configuración, conexión, migraciones iniciales cuando se apruebe modelo, autenticación y organización. Incremento 2: clientes/maestros, Gestión y borrador manual con propiedad, consulta compartida, auditoría y conflictos de versión.

Incremento 3: flujo completo de confirmación, operación, outbox y worker con simulador ARCA que permita probar éxito, rechazo y resultado incierto. Incremento 4: integración oficial verificada en homologación y reconciliación; probar ciclo real antes de producción.

Incremento 5: interpretación IA usando los mismos casos de uso y permisos. Incremento 6: actualización SISA, alertas, seguimiento oficial disponible y dashboard. Todos los elementos incluidos en el alcance deben estar resueltos antes de considerar completo el MVP; el orden no elimina requisitos.

Puertas pendientes: permisos de Gestión y maestros, autenticación, contrato ARCA, actualización SISA y modelo conceptual. Resolver cada una antes de implementar la funcionalidad afectada. El backlog se deriva de estos incrementos sin diseñar ahora todas las tablas.